

Univerzita Pardubice
Fakulta elektrotechniky a informatiky

Rozšíření webového prohlížeče pro interaktivní náhledy souborů

Tymofii Voronkin

Bakalářská práce

2026

ZADÁNÍ DIPLOMOVÉ PRÁCE

(PROJEKTU, UMĚLECKÉHO DÍLA, UMĚLECKÉHO VÝKONU)

Jméno a příjmení: [REDAKCE]
Osobní číslo: [REDAKCE]
Studijní program: N2646 Informační technologie
Studijní obor: Informační technologie
Název tématu: Úroveň podpory typografie v současných kancelářských editorech
Zadávající katedra: Katedra softwarových technologií

Z á s a d y p r o v y p r a c o v á n í :

V teoretické části budou popsána základní pravidla tvorby zejména odborného textu; včetně struktury textu, hladké, pořadové, technické a smíšené sazby.

Bude zhodnocena a porovnána úroveň podpory typografické sazby v současných kancelářských textových i tabulkových editorech (zejm. balíků od Microsoftu, LibreOffice a dokumenty Google).

Pro nepodporované vlastnosti budou uvedeny postupy, jak lze výsledku dosáhnout (nebo se mu přiblížit) alternativními způsoby (pokud je to možné).

Dále budou vytvořeny šablony pro psaní odborného textu (zejm. závěrečné práce), které budou splňovat typografická pravidla a budou odpovídat požadavkům UPa, a to pro editory MS Word, LibreOffice Writer a pro sázecí systém TeX ve variantách plain, LaTeX a ConTeXt. Šablony budou obsahovat nachystané stránkové, odstavcové, znakové (vyznačovací) i výčtové styly (včetně seznamů literatury, obrázků, obsahu apod.). Požadavky jednotlivých fakult budou konzultovány s příslušnými odpovědnými osobami. Šablony budou dodány ve variantách pro všechny fakulty a budou předány příslušným orgánům univerzity či fakult pro zveřejnění na webu či intranetu Univerzity. Šablona pro TeX bude vypracována jediná, potřebná nastavení (typ práce a fakulta) se bude volit příkazem či stylovým parametrem.

Rozsah grafických prací:
Rozsah pracovní zprávy: cca 40–50 stran
Forma zpracování diplomové práce: tištěná
Seznam odborné literatury: viz příloha

Vedoucí diplomové práce: Mgr. Tomáš Hudec
Katedra informačních technologií
Datum zadání diplomové práce: 31. října 2015
Termín odevzdání diplomové práce: 13. května 2016



prof. Ing. Simeon Karamazov, Dr.
děkan



prof. Ing. Antonín Kavička, Ph.D.
vedoucí katedry

V Pardubicích dne 15. listopadu 2015

Prohlašuji:

Tuto práci jsem vypracovala samostatně. Veškeré literární prameny a informace, které jsem v práci využila, jsou uvedeny v seznamu použité literatury.

Byla jsem seznámena s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorský zákon, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 9/2012, bude práce zveřejněna v Univerzitní knihovně a prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 17. 1. 2017

Tymofii Voronkin

Prohlášení o využití umělé inteligence

Při přípravě této práce byly využity nástroje umělé inteligence (zejména velké jazykové modely) v souladu s pravidly Univerzity Pardubice (Úroveň 2). Nástroje AI byly použity výhradně k dílčím úkonům:

- Asistence při návrhu softwarové architektury,
- Generování, úprava a kontrola částí zdrojového kódu (rozšíření prohlížeče),
- Gramatická korektura a stylistická úprava textu.

Za konečný obsah a výsledky práce nesu plnou zodpovědnost.

Poděkování

Rád bych poděkoval vedoucímu své bakalářské práce, Mgr. Tomáši Hudcovi, za odborné vedení, cenné rady a čas, který mi věnoval při konzultacích. Dále děkuji své rodině a přátelům za podporu během celého studia.

ANOTACE

Tato bakalářská práce se zabývá návrhem a implementací rozšíření pro webové prohlížeče (Google Chrome a Mozilla Firefox), které zvyšuje efektivitu prohlížení webových stránek. Rozšíření umožňuje okamžité zobrazení náhledů obrázků a PDF dokumentů pouhým najetím kurzoru myši na příslušný odkaz, čímž eliminuje nutnost stahování souborů a otevírání nových oken. Teoretická část popisuje architekturu rozšíření (Manifest V3), asynchronní programování v jazyce JavaScript a práci s knihovnou PDF.js. Praktická část detailně analyzuje implementaci logiky rozšíření, tvorbu uživatelského rozhraní s podporou filtrování domén pomocí regulárních výrazů a následné testování na reálných webových stránkách.

KLÍČOVÁ SLOVA

rozšíření prohlížeče, JavaScript, interaktivní náhledy, PDF.js, asynchronní programování, Manifest V3

TITLE

Web Browser Extension for Interactive File Previews

ANNOTATION

This bachelor thesis deals with the design and implementation of a web browser extension (for Google Chrome and Mozilla Firefox) that increases the efficiency of web browsing. The extension allows for instant previews of images and PDF documents simply by hovering the mouse cursor over the relevant link, eliminating the need to download files or open new windows. The theoretical part describes the architecture of extensions (Manifest V3), asynchronous programming in JavaScript, and working with the PDF.js library. The practical part analyzes in detail the implementation of the extension's logic, the creation of a user interface with support for domain filtering using regular expressions, and subsequent testing on real websites.

KEYWORDS

browser extension, JavaScript, interactive previews, PDF.js, asynchronous programming, Manifest V3

OBSAH

Seznam obrázků	9
Seznam tabulek	10
Seznam zkratek	11
Úvod	12
1 Teoretická část	14
1.1 Mechanismus vytváření rozšíření pro prohlížeče	14
1.2 Principy responzivních webových stránek s obrázky	15
1.3 Využívání událostí v JavaScriptu	16
1.4 JavaScriptové Fetch API	17
1.5 Používání Promises a asynchronní programování	18
1.6 Přehled existujících rozšíření pro náhledy a jejich hodnocení	19
2 Praktická část	21
2.1 Návrh a implementace rozšíření	21
2.2 Uživatelské rozhraní	22
2.3 Zobrazení náhledů PDF dokumentů	23
2.4 Testování na reálných webových stránkách	25
2.5 Uživatelská příručka (Instalace a konfigurace)	25
2.6 Limity stávajícího řešení a budoucí vývoj	27
Závěr	29
Použitá literatura	30
Seznam příloh	31
A Název přílohy A	32
B Soubor metadat XMP v PDF	33

SEZNAM OBRÁZKŮ

1.1	Ukázka konkurenčního rozšíření Hover Zoom+	19
1.2	Konfigurační rozhraní (Options) konkurenčního rozšíření Imagus	20
2.1	Architektura rozšíření a komunikace mezi jednotlivými komponentami	21
2.2	Uživatelské rozhraní vyskakovacího okna (Popup) v různých barevných motivech	22
2.3	Stránka s nastavením rozšíření, ukázka sekce pro správu domén a regulárních výrazů	23
2.4	Diagram sekvence procesu stahování a renderování PDF dokumentu	24
2.5	Zobrazení interaktivního náhledu PDF dokumentu na platformě Moodle	24
2.6	Náhled produktu na e-shopu Alza s využitím zobrazení obrázku v nejvyšším rozlišení	26
2.7	Plynulý náhled vektorového obrázku (SVG) na portálu UPCE	26

SEZNAM TABULEK

SEZNAM ZKRATEK

PDF Portable Document Format

Dlouhá zkratka Popisek zkratky

ÚVOD

V současné digitální éře představuje webový prohlížeč nejdůležitější bránu k informacím. Denně uživatelé projdou desítky až stovky webových stránek, přičemž se neustále setkávají s obrovským množstvím hypertextových odkazů, obrázků a dokumentů. Tradiční způsob interakce s tímto obsahem obvykle vyžaduje aktivní kliknutí na odkaz a načtení nové stránky, nebo stažení souboru na lokální disk před jeho samotným zobrazením. Tento proces je nejen časově náročný, ale také narušuje plynulost práce (tzv. *workflow*) a zbytečně plní úložiště uživatele staženými soubory, které byly potřeba jen pro rychlé nahlédnutí.

S narůstajícím tempem konzumace obsahu roste i poptávka po efektivnějších způsobech procházení webu. Jedním z možných řešení tohoto problému jsou rozšíření pro webové prohlížeče (tzv. *browser extensions*), která dokážou modifikovat chování webových stránek a přidávat nové funkce přímo do uživatelského rozhraní prohlížeče.

Cílem této bakalářské práce je navrhnout, naimplementovat a otestovat právě takové softwarové řešení – moderní rozšíření pro webové prohlížeče (zaměřené na platformy Google Chrome a Mozilla Firefox), které uživatelům umožní okamžité zobrazení interaktivních náhledů obrázků a dokumentů (zejména formátu PDF). Rozšíření funguje na principu aktivace při pouhém najetí kurzoru myši (událost *hover*) nad vybraný webový element, čímž eliminuje nutnost klikání či otevírání nových panelů.

Tato práce je rozdělena na dvě hlavní části. V teoretické části (viz kapitola 1) jsou detailně rozebrány mechanismy a principy nezbytné pro vývoj takového rozšíření. Čtenář je seznámen s architekturou rozšíření prohlížečů postavenou na specifikaci Manifest V3, technikami responzivního webového designu (s důrazem na práci s obrázky pomocí atributů `srcset` a `sizes`) a obsluhou asynchronních událostí v jazyce JavaScript. Důležitá pozornost je věnována také moderním přístupům získávání dat přes rozhraní Fetch API a využití *Promises*. Na závěr teoretické části je provedena analýza a zhodnocení existujících konkurenčních řešení, což slouží jako východisko pro vlastní implementaci.

Praktická část (viz kapitola 2) pak popisuje konkrétní realizaci navrženého rozšíření. Zabývá se softwarovou architekturou projektu, implementací logiky detekce odkazů a tvorbou uživatelského rozhraní, které mimo jiné nabízí možnost pokročilé konfigurace domén pomocí regulárních výrazů (systém *whitelist/blacklist*). Významná podkapitola je věnována integraci knihovny PDF.js pro bezpečné a plynulé renderování PDF doku-

mentů přímo v prohlížeči. Práce je zakončena kapitolou shrnující testování rozšíření v reálných podmínkách na frekventovaných webových stránkách a zhodnocením dosažených výsledků.

1 TEORETICKÁ ČÁST

V této části je teoreticky popsán mechanismus fungování webových rozšíření a použité technologie.

1.1 Mechanismus vytváření rozšíření pro prohlížeče

Rozšíření prohlížeče (anglicky *browser extension*) je malý softwarový modul, který přizpůsobuje zážitek z prohlížení webu. Rozšíření umožňují uživatelům přizpůsobit funkcionalitu a chování prohlížeče individuálním potřebám nebo preferencím Mozilla Foundation, 2025b. Moderní rozšíření jsou budována pomocí standardních webových technologií, tedy jazyka HTML, kaskádových stylů (CSS) a jazyka JavaScript Flanagan, 2020. Většina současných prohlížečů, včetně Google Chrome, Mozilla Firefox, Microsoft Edge a Safari, podporuje unifikovaný standard WebExtensions API, díky kterému lze napsat kód jednou a s minimálními úpravami jej zkompileovat pro různé platformy Frisbie, 2025.

Historicky byla většina rozšíření postavena na architektuře Manifest V2. Vzhledem k rostoucím obavám o soukromí uživatelů a bezpečnost však společnost Google představila novou specifikaci Manifest V3, která se stala novým průmyslovým standardem. Hlavním rozdílem mezi V2 a V3 je přesun od tzv. *Background Pages* (stránek na pozadí, které běžely neustále a zabíraly operační paměť) k *Service Workerům*, které jsou probouzeny pouze na základě specifických událostí. Tím se výrazně snižuje paměťová náročnost rozšíření. Další klíčovou změnou je přísnější bezpečnostní model, který zakazuje spouštění vzdáleně hostovaného kódu (remote code execution) a omezuje pravidla pro blokování síťových požadavků přes `webRequest` API, což nutí vývojáře využívat nové deklarativní API Google, 2025.

Klíčovým stavebním kamenem každého rozšíření je konfigurační soubor s názvem `manifest.json`. Tento soubor definuje základní metadata aplikace (název, verzi, popis), požadovaná oprávnění (např. přístup k aktivní záložce přes `activeTab` nebo oprávnění k ukládání dat přes `storage`) a deklaruje komponenty, ze kterých se rozšíření skládá.

Architektura běžného rozšíření se skládá z několika oddělených, avšak vzájemně komunikujících komponent Frisbie, 2025:

- **Content Scripts (Skripty obsahu):** Tyto JavaScriptové soubory jsou injektovány přímo do kontextu webových stránek, které uživatel navštíví. Mohou číst

a upravovat Document Object Model (DOM) dané stránky, což je nezbytné pro detekci prvků (např. odkazů nebo obrázků) a modifikaci uživatelského rozhraní přímo na prohlíženém webu. Sdílejí DOM s originální stránkou, ale jejich spouštěcí prostředí a proměnné jsou izolované.

- **Background Scripts (Skripty na pozadí):** V rámci Manifestu V3 jsou realizovány prostřednictvím tzv. *Service Workerů*. Běží nezávisle na životním cyklu webové stránky, reagují na systémové události prohlížeče (jako je např. kliknutí na ikonu rozšíření, instalace aktualizace) a mohou udržovat dlouhodobý stav. Tyto skripty nemají přímý přístup k DOMu žádné stránky, a proto musí s Content Skripty komunikovat přes mechanismus zpráv (Message Passing) Google, 2025.
- **Uživatelské rozhraní (Popup a Options):** Většina rozšíření disponuje viditelným grafickým rozhraním. *Popup* se zobrazí při kliknutí na ikonu rozšíření v liště prohlížeče a slouží k rychlým interakcím. Stránka *Options* (Možnosti) poskytuje komplexnější prostředí pro konfiguraci rozšíření, přičemž uživatelská nastavení se obvykle ukládají pomocí `chrome.storage` API, odkud je následně mohou asynchronně načítat ostatní komponenty.

Omezení přístupových práv (*Permissions*) představuje jeden z nejdůležitějších bezpečnostních konceptů při vývoji rozšíření. Na rozdíl od běžných webových stránek mohou rozšíření po udělení souhlasu přistupovat k citlivým funkcím operačního systému prohlížeče, např. obcházet pravidla CORS (Cross-Origin Resource Sharing) pro asynchronní požadavky na vzdálené servery Mozilla Foundation, 2025b. Toto je klíčové pro aplikace stahující dodatečný obsah na pozadí, aniž by došlo k blokování požadavku ze strany bezpečnostních mechanismů samotné prohlížené stránky.

1.2 Principy responzivních webových stránek s obrázky

Při vývoji rozšíření pro zobrazování náhledů obrázků je klíčové porozumět moderním technikám vkládání grafiky do HTML dokumentu. V minulosti vývojáři spoléhali výhradně na atribut `src` u značky `<img>`. S nástupem mobilních zařízení s displeji o vysoké hustotě pixelů (např. Retina displeje) však vznikla potřeba responzivního načítání obrázků Frain, 2020.

Moderní specifikace HTML WHATWG (Apple, Google, Mozilla, Microsoft), 2025 zavedla dva klíčové atributy pro element `<img>`, a to `srcset` a `sizes`, případně samostatný element `<picture>`. Atribut `srcset` obsahuje čárkou oddělený seznam adres obrázků spolu s jejich šířkou (např. `image-800w.jpg 800w`). Atribut `sizes` pak definuje, jaký prostor bude obrázek na obrazovce zabírat v závislosti na tzv. *media queries* (např. `(max-width: 600px) 100vw, 50vw`). Prohlížeč následně sám asynchronně vyhodnotí velikost okna, hustotu pixelů displeje a z atributu `srcset` stáhne nejvhodnější obrázek Mozilla Foundation, 2025a.

Z hlediska vývoje rozšíření, které má po najetí myši zobrazit zvětšený náhled obrázku, to znamená, že se nelze spolehnout pouze na hodnotu atributu `src`, protože ta může obsahovat pouze *fallback* (záložní obrázek) v nízkém rozlišení. Namísto toho musí rozšiřující skript analyzovat atribut `srcset` nebo nadřazené značky `<source>` a extrahovat URL adresu obrázku s nejvyšším možným rozlišením.

1.3 Využívání událostí v JavaScriptu

Aby rozšíření dokázalo reagovat na činnost uživatele – konkrétně na pohyb myši nad vybranými elementy – je nutné využít systém událostí (tzv. *Event Handling*) v rámci DOM modelu. JavaScript komunikuje s HTML dokumentem asynchronně prostřednictvím posluchačů událostí (`EventListeners`) Flanagan, 2020.

Pro účely náhledového rozšíření jsou klíčové události související s pohybem kurzoru Mozilla Foundation, 2025c:

- **mouseover** a **mouseout**: Tyto události jsou vyvolány, když kurzor vstoupí do oblasti elementu, respektive ji opustí. Jejich specifkem je, že tzv. *probublávají* (Event Bubbling) směrem nahoru strukturou DOMu k nadřazeným elementům. Toho lze využít pro efektivní zachytávání událostí na celém dokumentu (tzv. *Event Delegation*) namísto vázání samostatného posluchače na každý jednotlivý obrázek či odkaz na stránce.
- **mouseenter** a **mouseleave**: Podobné jako předchozí, avšak s tím rozdílem, že neprobublávají. K vyvolání dojde pouze tehdy, když kurzor myši fyzicky překročí hranici elementu, na kterém je posluchač navázán Mozilla Foundation, 2025c.
- **mousemove**: Vyvolává se opakovaně po celou dobu, kdy se kurzor pohybuje uvnitř elementu. V kontextu rozšíření se využívá k průběžnému přepočítávání pozice (X ,

Y souřadnic) náhledového okna tak, aby plynule následovalo pohyb uživatelské myši.

Vzhledem k četnosti spouštění události `mousemove` je při implementaci nezbytné dbát na výkon a využívat optimalizační techniky. Pokud by se při každém pohnutí myši nad prvkem asynchronně spouštěly složité výpočty nebo překreslování DOMu, mohlo by docházet k zamrznutí uživatelského rozhraní prohlížeče Flanagan, 2020.

Nejčastěji se využívají návrhové vzory *debounce* a *throttle*. Vzor *debounce* zajišťuje, že se určitá funkce spustí až po uplynutí definované doby nečinnosti od posledního vyvolání události. To je ideální pro detekci záměrného najetí myši (tzv. *intent to hover*), nikoliv pouhého přejetí přes odkaz během posouvání stránky. Příklad takové implementace, kde je k zobrazení náhledu využit časovač (timeout), ukazuje výpis 1.

```
1 function debounce(func, wait) {  
2     let timeout;  
3     return function(...args) {  
4         clearTimeout(timeout);  
5         timeout = setTimeout(() => func.apply(this, args), wait);  
6     };  
7 }  
8  
9 // Použití pro zpožděné zobrazení náhledu po 500 ms  
10 const linkElement = document.querySelector('a');  
11 linkElement.addEventListener('mouseover', debounce(showPreview, 500));
```

Zdrojový kód 1: Příklad jednoduché implementace funkce `debounce` pro odložení náhledu

1.4 JavaScriptové Fetch API

Pro načtení obsahu cílového odkazu (např. stažení PDF souboru) před jeho zobrazením v náhledovém okně slouží asynchronní HTTP požadavky. V minulosti byl za tímto účelem využíván objekt `XMLHttpRequest` (AJAX). Moderní vývoj však zcela přechází na rozhraní **Fetch API**, které poskytuje robustnější, flexibilnější a logičtější způsob pro získávání síťových zdrojů Flanagan, 2020.

Globální metoda `fetch()` iniciuje proces stahování zdroje ze sítě a na rozdíl od starších řešení vrací rovnou objekt typu `Promise` (slib), čímž elegantně řeší problémy spojené s vnořováním asynchronních *callback* funkcí. Použití Fetch API v kombinaci s *Service Workery* rozšíření umožňuje elegantně stahovat bloky dat (např. formou datových proudů – *Streams*) i z domén, které by za normálních okolností blokovala politika stejného původu (Same-Origin Policy), a tato data následně dynamicky injectovat do prohlížené webové stránky formou HTML či Canvas elementů.

1.5 Používání Promises a asynchronní programování

Běžové prostředí jazyka JavaScript je ve své podstatě jednovláknové (*single-threaded*). Jakákoliv zdlouhavá operace (např. stahování velkého PDF dokumentu) by v synchronním režimu zablokovala hlavní vlákno a způsobila zamrznutí celého uživatelského rozhraní prohlížeče. K řešení tohoto problému slouží asynchronní programování Flanagan, 2020.

Základním kamenem moderní asynchronní logiky v JavaScriptu je objekt `Promise` (česky příslib nebo slib). `Promise` reprezentuje hodnotu, která v daném okamžiku ještě nemusí existovat, ale bude dostupná někdy v budoucnosti Mozilla Foundation, 2025d. Tento objekt se může nacházet ve třech stavech:

- **Pending (čekající):** Výchozí stav, asynchronní operace ještě nebyla dokončena.
- **Fulfilled (splněný):** Operace byla úspěšně dokončena a `Promise` obsahuje výslednou hodnotu.
- **Rejected (zamítnutý):** Operace selhala (např. chyba sítě, soubor nenalezen) a `Promise` obsahuje důvod selhání (objekt chyby).

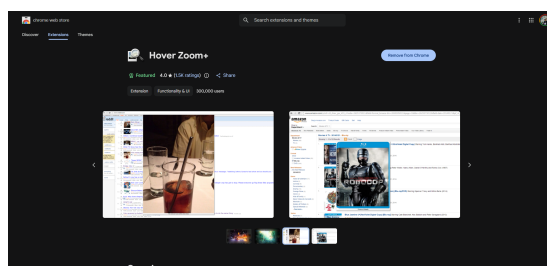
I když API samotných *Promises* (metody `.then()` a `.catch()`) výrazně zlepšilo čitelnost kódu oproti starším metodám, novější specifikace ECMAScript představily syntaktický cukr v podobě klíčových slov `async` a `await`. Tato syntaxe umožňuje psát asynchronní kód tak, aby vypadal a choval se jako klasický kód synchronní, což zásadně snižuje složitost logiky při vývoji komplexních rozšíření.

1.6 Přehled existujících rozšíření pro náhledy a jejich hodnocení

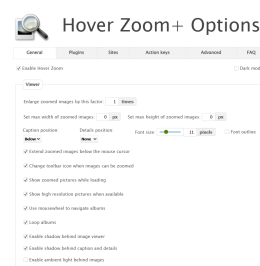
Koncept interaktivních náhledů pomocí najetí myši (hover) není v prostředí webových prohlížečů novinkou. Existuje několik zaběhnutých řešení, mezi nejznámější patří rozšíření *Imagus* a *Hover Zoom+*. Obě tato řešení však trpí určitými nedostatky, které odůvodňují vývoj vlastního, moderního nástroje.

Rozšíření Hover Zoom+

Rozšíření **Hover Zoom+** (viz obrázek 1.1) je jedním z historicky nejznámějších nástrojů v této kategorii. Původní verze (Hover Zoom) měla v minulosti vážné bezpečnostní problémy spojené se sledováním uživatelů a vkládáním škodlivého kódu (spyware), což vedlo k narušení důvěry komunity a jejímu odstranění z obchodů s doplňky. Současná verze se sice pyšní otevřeným zdrojovým kódem (open-source), avšak z uživatelského hlediska trpí zastaralým designem a nepřehledným rozhraním s obrovským množstvím komplikovaných nastavení, ve kterých se běžný uživatel snadno ztratí. Navíc podpora dokumentů je velmi omezená a zcela chybí vestavěná integrace pro náhledy PDF souborů bez nutnosti instalace dalších externích modulů.



(a) Stránka v Chrome Web Store



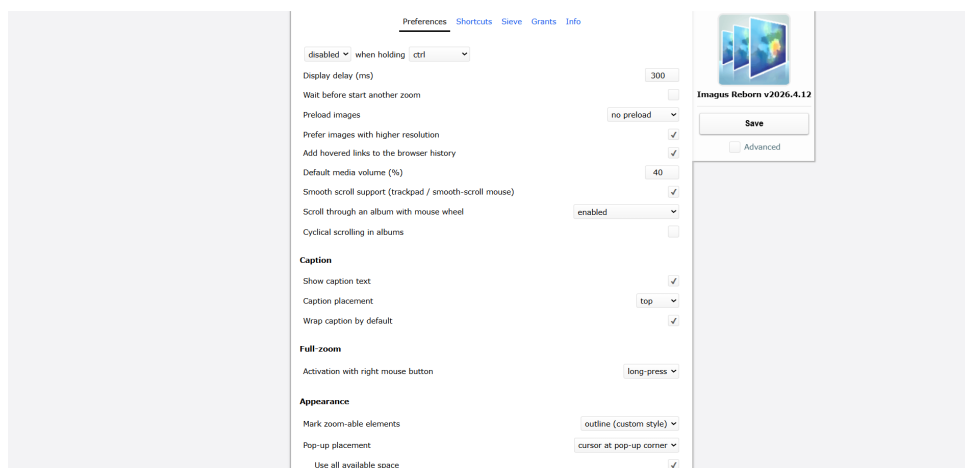
(b) Zastaralé uživatelské rozhraní

Obrázek 1.1: Ukázka konkurenčního rozšíření Hover Zoom+

Rozšíření Imagus

Rozšíření **Imagus** (viz obrázek 1.2) představuje podobný nástroj. Je sice populární a funkční pro široké spektrum obrázkových galerií, avšak jeho vývoj v posledních letech do značné míry ustrnul (tzv. abandonware). Kódová základna byla původně optimalizována pro Manifest V2, což s sebou přináší riziko nefunkčnosti v moderních prohlížečích po

plném přechodu na specifikaci V3. I zde platí, že chybí podpora nativního renderování vícestránkových PDF dokumentů, a stejně jako Hover Zoom+ se vyznačuje složitou konfigurací pravidel (Sieve), která vyžaduje psaní složitých regulárních výrazů i pro zcela banální úkoly.



Obrázek 1.2: Konfigurační rozhraní (Options) konkurenčního rozšíření Imagus

Srovnání a přidaná hodnota vlastního řešení

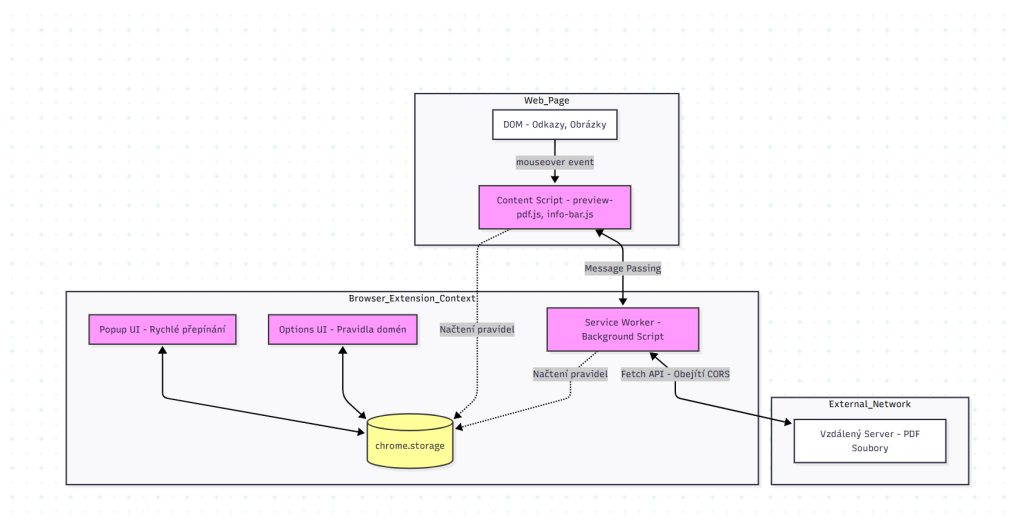
Zásadním nedostatkem naprosté většiny současných rozšíření je omezení čistě na bitmapovou grafiku (JPEG, PNG, GIF) či krátká videa (MP4, WebM). Implementace vlastního rozšíření s integrovanou knihovnou *PDF.js* tak přináší signifikantní přidanou hodnotu. Uživatel získává moderní, bezpečné a snadno ovladatelné rozhraní postavené na bezpečném standardu Manifest V3, které umožňuje rychlý náhled vícestránkových dokumentů přímo z e-learningových systémů či univerzitních portálů, a to bez nutnosti jakéhokoliv stahování souborů na lokální disk.

2 PRAKTICKÁ ČÁST

V praktické části je popsán proces návrhu, implementace a následného testování vytvořeného rozšíření.

2.1 Návrh a implementace rozšíření

Architektura vyvíjeného rozšíření, nazvaného *Interactive Previews*, je navržena modulárně a striktně odděluje jednotlivé logické celky. Jádrem projektu je definováno v souboru `manifest.json` (verze 3), který deklaruje potřebná práva, primárně `activeTab` pro přístup k aktuální stránce a `storage` pro ukládání uživatelského nastavení. Schéma komunikace mezi komponentami znázorňuje obrázek 2.1.



Obrázek 2.1: Architektura rozšíření a komunikace mezi jednotlivými komponentami

Kódová základna je strukturována do několika klíčových adresářů:

- `src/content/` obsahuje skripty injektované do webových stránek (`content.js`, `preview-image.js`, `preview-pdf.js` a `info-bar.js`). Tyto skripty primárně naslouchají události `mouseover` na odkazech a obrázcích. Po uplynutí definovaného zpoždění (tzv. *hover delay*) se asynchronně vytvoří a zobrazí plovoucí kontejner s náhledem.
- `src/background/` zajišťuje běh na pozadí pomocí Service Workeru. Tento modul slouží především jako proxy pro HTTP požadavky (Fetch), čímž řeší restriktce CORS při stahování vzdálených PDF souborů do náhledového okna.

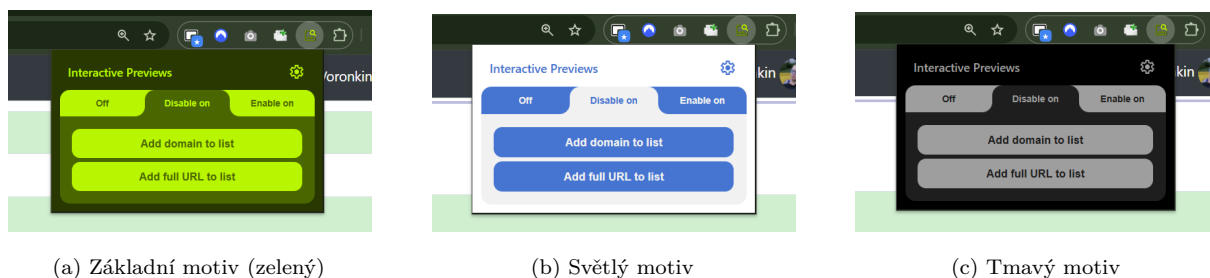
- `src/common/` sdružuje pomocné funkce a konstanty využívané napříč celým rozšířením.

Vykreslovací logika počítá i s optimalizací výkonu. Zobrazený náhledový element (tzv. informační panel neboli *Info Bar*) poskytuje uživateli základní metadata o souboru a ovládací prvky, aniž by blokoval překreslování původní webové stránky. Pokud uživatel myši opustí oblast odkazu či náhledu (událost `mouseleave`), element se okamžitě bezpečně odstraní z DOMu, čímž se uvolní alokovaná operační paměť.

2.2 Uživatelské rozhraní

Z pohledu uživatele se rozšíření skládá ze dvou grafických rozhraní: z vyskakovacího okna (Popup) a stránky s podrobným nastavením (Options). Grafika je navržena v souladu s moderními designovými trendy s důrazem na čistotu a srozumitelnost.

Vyskakovací okno (`src/popup/`) se zobrazí po kliknutí na ikonu rozšíření v horní liště prohlížeče. Slouží jako rychlý přepínač umožňující okamžité zapnutí či vypnutí funkcionality na právě prohlíženém webu.



(a) Základní motiv (zelený)

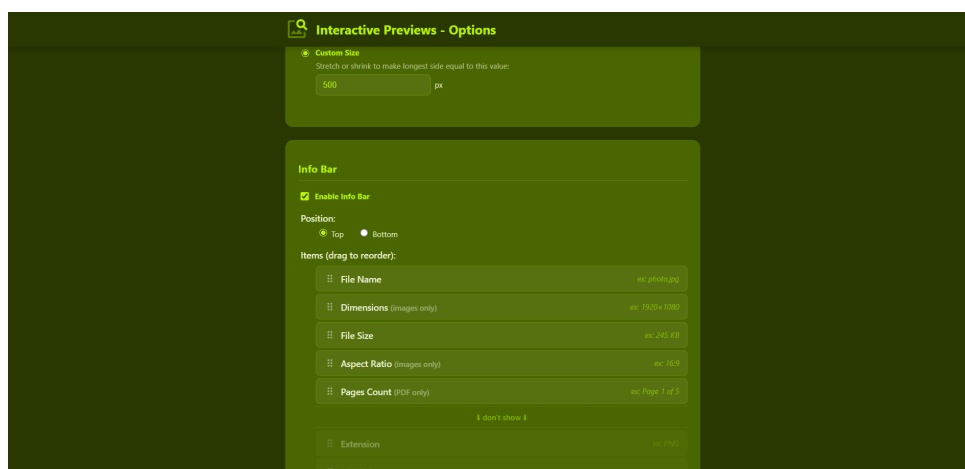
(b) Světlý motiv

(c) Tmavý motiv

Obrázek 2.2: Uživatelské rozhraní vyskakovacího okna (Popup) v různých barevných motivech

Stránka s nastavením (`src/options/`) nabízí pokročilou konfiguraci. Stěžejní funkcí je možnost detailní správy chování na specifických doménách prostřednictvím modulu pro filtrování. Uživatel si může vybrat ze tří režimů: *Off* (zcela vypnuto), *Disable on* (zakázáno na vybraných doménách – blacklist) a *Enable on* (povoleno pouze na vybraných doménách – whitelist). Výrazným prvkem tohoto rozhraní je podpora **regulárních výrazů (Regex)**. Díky tomu mohou pokročilí uživatelé definovat složitá pravidla pokrývající celé sítě webů (např. `.*.wikipedia.org`). Veškerá tato nastavení se ihned ukládají pomocí asynchronního API `chrome.storage.sync`,

což zajišťuje synchronizaci napříč všemi zařízeními daného uživatele přihlášeného k účtu prohlížeče.

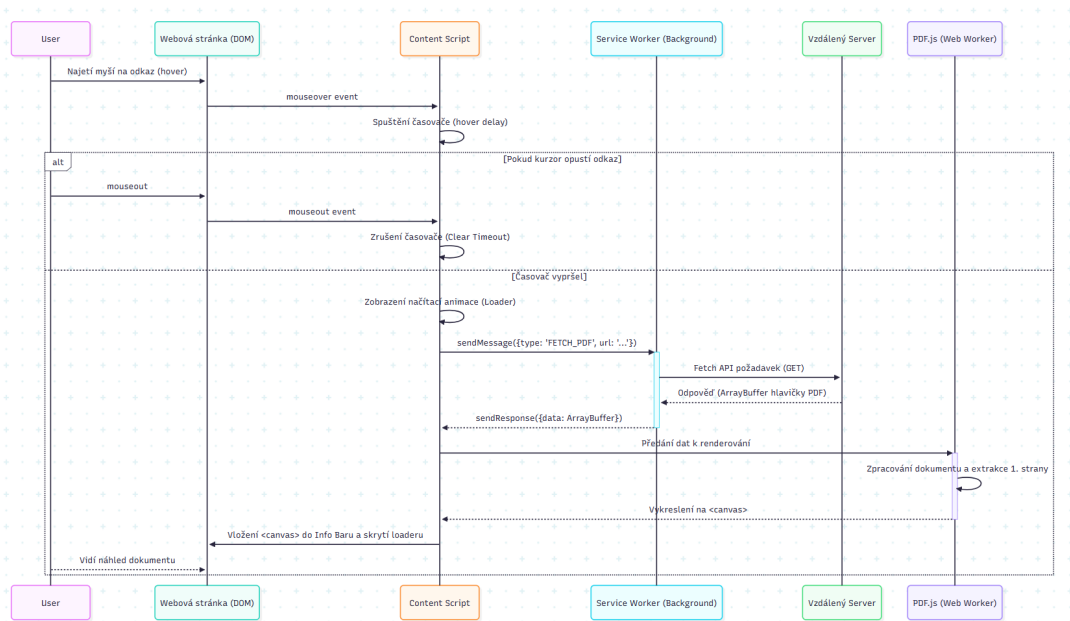


Obrázek 2.3: Stránka s nastavením rozšíření, ukázka sekce pro správu domén a regulárních výrazů

2.3 Zobrazení náhledů PDF dokumentů

Nejkomplexnější technickou výzvou celého projektu byla implementace interaktivních náhledů dokumentů ve formátu PDF. Vzhledem k tomu, že prohlížeče neposkytují nativní a snadno dostupné API pro extrakci obsahu z PDF přímo do kontextu webové stránky, bylo přistoupeno k integraci populární open-source knihovny **PDF.js** vyvíjené společností Mozilla Mozilla, 2025.

Proces zobrazení probíhá v několika fázích. Modul `preview-pdf.js` nejprve detekuje, že odkaz, nad kterým se nachází kurzor, směřuje na soubor s příponou `.pdf`. Následně vyvolá zprávu na *Background Script*, který pomocí Fetch API asynchronně stáhne hlavičku a první stranu daného dokumentu ve formátu `ArrayBuffer`. Tato data jsou předána knihovně *PDF.js*, jejíž výpočetně náročné jádro (`pdf.worker.min.js`) běží odděleně v rámci Web Workeru, aby nedošlo k zablokování hlavního vlákna prohlížeče. Výsledná rastrová podoba první strany dokumentu je následně vykreslena na dynamicky vytvořený element `<canvas>`, který je bezpečně vložen do informačního panelu náhledu. Celý proces od zachycení události po vykreslení dokumentu je schematicky znázorněn na diagramu sekvence (viz obrázek 2.4).



Obrázek 2.4: Diagram sekvence procesu stahování a renderování PDF dokumentu

Stav odevzdání úkolu

Stav odevzdání úkolu	Odesláno k hodnocení
Stav hodnocení	Nehodnoceno
Zbývá	Úkoly byly odevzdány 7 dnů 9 hodin včas
Naposledy změněno	pátek, 1. května 2026, 14:41
Odevzdané soubory	 Dokumentace, prezentace a rétorika - Es
Komentář studenta	Komentáře (0)

◀ Odprezentuj jako řečnická superstar

Přejít na...

Dokumentace, prezentace a rétorika - Esej - Tymofii Voronkin.pdf - Page 1 of 1

BDPR - Dokumentace, prezentace a rétorika

Esaj na téma:

Retorika, k čemu ji potřebujeme, k čemu potřebujeme jazyk, co si o využití jazyka a retoriky sami myslíme

Autor: Tymofii Voronkin

Jazyk je pro nás v dnešní době něco naprosto nezastupitelného. Bez něho by pravděpodobně nemohli existovat žádné společnosti. Jazyk je totiž tím, co nás spojuje a umožňuje nám sdělovat své myšlenky a emoce. Retorika, k čemu ji potřebujeme, k čemu potřebujeme jazyk, co si o využití jazyka a retoriky sami myslíme. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé.

Když se ale řekne, že retorika je jen o slovech, je to velká chyba. Retorika je o mnohem více. Je o umění přesvědčovat, o umění sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé.

Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé.

Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé. Retorika nám pomáhá sdělovat své myšlenky tak, aby byly srozumitelné a působivé.

Obrázek 2.5: Zobrazení interaktivního náhledu PDF dokumentu na platformě Moodle

2.4 Testování na reálných webových stránkách

Pro ověření spolehlivosti a uživatelské přívětivosti bylo rozšíření podrobeno sadě testů na reálných a frekventovaných webových portálech. Testování bylo navrženo podle předem definovaných scénářů (tzv. *Test Cases*), které ověřovaly správné chování na různých typech obsahu.

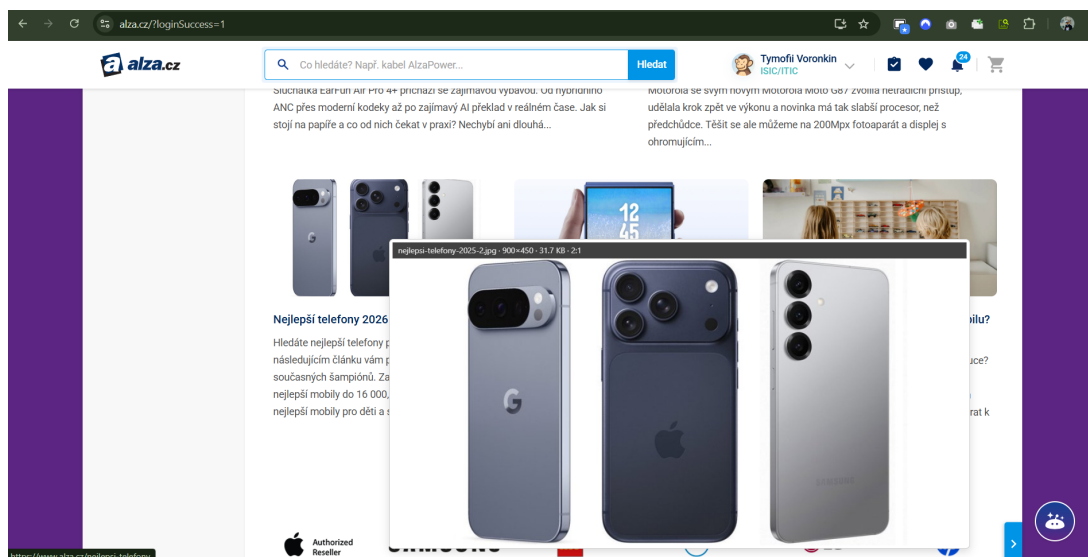
Mezi hlavní testovací scénáře patřily:

- **Detekce obrázků ve vysokém rozlišení:** Ověření, zda rozšíření ignoruje náhledové miniatury v nízké kvalitě a přes atributy `srcset` nebo `data-src` vyhledává a zobrazuje plné verze.
- **Renderování PDF z univerzitních systémů:** Zátěžový test zkoumající chování na objemných dokumentech (syllaby, přednášky v Moodle, zadání v IS STAG). Sledoval se čas potřebný pro stažení dat (včetně případných restrikcí CORS) a plynulost vykreslování první strany.
- **Ochrana proti nechtěnému spuštění (Accidental Hover):** Simulace běžného čtení a rychlého posunu kurzoru přes obrazovku. Očekávaným chováním je, že časovač s prodlevou (*hover delay*) spolehlivě zabrání okamžitému zobrazení náhledu a ušetří tak výpočetní i síťové zdroje.

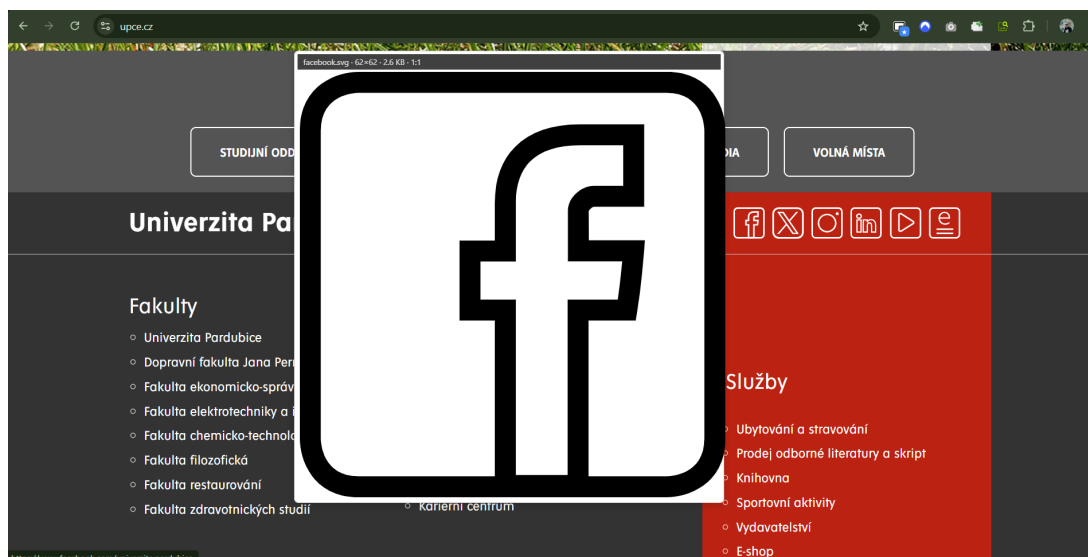
Při testování na portálu *Wikipedia* a e-shopu *Alza* (viz obrázek 2.6) rozšíření úspěšně extrahovalo verze obrázků ve vysokém rozlišení z atributů `srcset`, přičemž plovoucí okno plynule následovalo kurzor myši bez tzv. artefaktů překreslování (*screen tearing*). Na univerzitním informačním systému a hlavních stránkách univerzity (viz obrázek 2.7) pak rozšíření spolehlivě zachytávalo odkazy směřující na PDF a SVG soubory, stahovalo je na pozadí a s minimální latencí vykreslovalo první strany dokumentů, a to i u vizuálně komplexních syllabů předmětů. Proti nechtěnému otevírání náhledů při rychlém přejetí myší byla úspěšně otestována implementace časovače s konstantním zpožděním 500 milisekund.

2.5 Uživatelská příručka (Instalace a konfigurace)

Vzhledem k tomu, že rozšíření není v době psaní práce publikováno ve veřejných obchodech s doplňky (Chrome Web Store / Firefox Add-ons), probíhá jeho instalace prostřednictvím vývojářského režimu.



Obrázek 2.6: Náhled produktu na e-shopu Alza s využitím zobrazení obrázku v nejvyšším rozlišení



Obrázek 2.7: Plynulý náhled vektorového obrázku (SVG) na portálu UPCE

Postup instalace pro prohlížeče na bázi Chromium (Google Chrome, Edge, Brave):

1. Uživatel stáhne zdrojové kódy ve formátu ZIP a rozbalí je do zvoleného adresáře.
2. V prohlížeči otevře stránku se správou rozšíření zadáním adresy `chrome://extensions/`.
3. V pravém horním rohu aktivuje přepínač **Režim pro vývojáře** (Developer mode).
4. Klikne na tlačítko **Načíst rozbalené** (Load unpacked) a vybere složku `src` z rozbaleného archivu.
5. Rozšíření je tímto nainstalováno a jeho ikona se objeví na hlavní liště.

Po úspěšné instalaci může uživatel kliknout na ikonu rozšíření a otevřít vyskakovací okno (Popup) pro okamžitou správu stavu, nebo přejít do nastavení (Options) kliknutím pravým tlačítkem myši na ikonu a volbou *Možnosti*. Zde lze přidávat vlastní domény do whitelistu či blacklistu.

2.6 Limity stávajícího řešení a budoucí vývoj

Přestože navržené řešení úspěšně splňuje definované cíle, existují v současné verzi určité technické a funkční limity, které by mohly být předmětem dalšího vývoje:

- **Podpora kancelářských formátů:** V současnosti systém nativně renderuje primárně formáty běžných obrázků (JPEG, PNG, SVG, WebP) a PDF dokumenty. Rozšíření o podporu kancelářských dokumentů typu `.docx`, `.xlsx` nebo `.pptx` by výrazně zvýšilo užitnou hodnotu, avšak vyžadovalo by integraci dalších komplexních knihoven nebo využití externích cloudových API (např. Google Docs Viewer), což s sebou nese rizika z hlediska ochrany soukromí.
- **Výkon u gigantických souborů:** Ačkoliv integrace *PDF.js* probíhá přes nezávislé vlákno (Web Worker), u extrémně objemných PDF souborů (stovky megabytů dat s komplexní vektorovou grafikou) může docházet ke zpoždění při stahování a úvodním renderování první strany. Zavedení inteligentního streamování a využití HTTP hlaviček `Range` by tento proces optimalizovalo.
- **Přehrávání multimédií:** Konkurenční řešení často podporují náhledy animovaných GIFů nebo videí. Současná verze rozšíření se videím záměrně vyhýbá kvůli snížení paměťové náročnosti a zamezení rušivému přehrávání zvuku (autoplay), avšak do budoucna lze implementovat volitelný modul pro ztlumené přehrávání oblíbených video formátů.

V dalším vývoji je rovněž v plánu publikovat rozšíření v oficiálních obchodech doplňků, čímž by se zjednodušil proces instalace pro běžné uživatele a zajistila by se automatická aktualizace softwaru.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout a implementovat moderní rozšíření pro webové prohlížeče, které zefektivní práci s online obsahem tím, že uživatelům umožní okamžité zobrazení náhledů obrázků a PDF dokumentů pouhým najetím kurzoru myši.

V teoretické části byly popsány mechanismy asynchronního programování, zpracování událostí v rámci DOM struktury a limity současných technologických standardů. Tyto poznatky byly následně aplikovány v praktické části při vývoji samotného softwarového modulu. Výsledkem je plně funkční, responzivní a výkonnostně optimalizované rozšíření *Interactive Previews*, které bezpečně integruje knihovnu PDF.js pro vykreslování vícestránkových dokumentů a nabízí uživateli rozsáhlé možnosti personalizace prostřednictvím regulárních výrazů (filtrování domén).

Testování na produkčních webech potvrdilo, že navržené řešení eliminuje nutnost neustálého klikání a stahování souborů za účelem rychlé inspekce jejich obsahu, čímž prokazatelně šetří čas a zvyšuje plynulost prohlížení webu. Z hlediska budoucího vývoje se nabízí možnost integrace podpory dalších komplexních kancelářských formátů, například DOCX či XLSX, a implementace analytických nástrojů pro měření průměrné časové úspory koncových uživatelů.

POUŽITÁ LITERATURA

- FLANAGAN, David, 2020. *JavaScript: The Definitive Guide. Master the World's Most-Used Programming Language*. 7. vyd. Cambridge: O'Reilly. ISBN 978-1-4919-5200-9.
- FRAIN, Ben, 2020. *Responsive web design with HTML5 and CSS: Develop future-proof responsive websites using the latest HTML5 and CSS techniques*. 3. vyd. Birmingham: Packt. ISBN 978-1-83921-156-0.
- FRISBIE, Matt, 2025. *Building Browser Extensions: Create Modern Extensions for Chrome, Safari, Firefox, and Edge*. 2. vyd. Apress Berkeley. ISBN 979-8-8688-1593-5.
- GOOGLE, 2025. *API reference* [online]. Chrome Extensions, 2025-08-05. [cit. 2025-10-30]. URL: <https://developer.chrome.com/docs/extensions/reference/api>.
- MOZILLA, 2025. *PDF.js* [online]. GitHub, 2025-10-05. [cit. 2025-10-30]. URL: <https://github.com/mozilla/pdf.js>.
- MOZILLA FOUNDATION, 2025a. *: The Image Embed element* [online]. Mozilla Developer Network (MDN) Web Docs, 2025-10-10. [cit. 2025-10-30]. URL: <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/img>.
- MOZILLA FOUNDATION, 2025b. *Browser extensions* [online]. Mozilla Developer Network (MDN) Web Docs, 2025-07-17. [cit. 2025-10-30]. URL: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions>.
- MOZILLA FOUNDATION, 2025c. *Mouse events* [online]. Mozilla Developer Network (MDN) Web Docs, 2025-09-23. [cit. 2025-10-30]. URL: https://developer.mozilla.org/en-US/docs/Web/API/Element#mouse_events.
- MOZILLA FOUNDATION, 2025d. *Using promises* [online]. Mozilla Developer Network (MDN) Web Docs, 2025-09-18. [cit. 2025-10-30]. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise.
- WHATWG (APPLE, GOOGLE, MOZILLA, MICROSOFT), 2025. *HTML: Living Standard* [online]. 2025-10-25. [cit. 2025-10-31]. URL: <https://html.spec.whatwg.org/>.

SEZNAM PŘÍLOH

A	Název přílohy A	32
B	Soubor metadat XMP v PDF	33

Příloha A: Název přílohy A

Popis přílohy A.

Příloha B: Soubor metadat XMP v PDF

Obsah souboru s metadaty ve formátu XMP. Vyžadováno pro PDF/A.

```
\Title{Rozšíření webového prohlížeče pro interaktivní náhledy souborů}
```

```
\Author{Tymofii Voronkin}
```

```
\Keywords{rozšíření prohlížeče\sep JavaScript\sep interaktivní náhledy\sep PDF.js}
```

```
\Publisher{Univerzita Pardubice}
```

```
\Subject{Bakalářská práce}
```

```
\Language{cs}
```